Student Skills Development Management System - Source Code

File: main.py

```
from fastapi import FastAPI, Request, Depends, HTTPException, status
from fastapi.staticfiles import StaticFiles from fastapi.templating
import Jinja2Templates from fastapi.responses import HTMLResponse,
RedirectResponse from database.database import get_db, db from
routers import auth, student, admin from routers.auth import
get_password_hash from datetime import datetime import os  app =
FastAPI(title="Student Skills Development Management System")
 app.mount("/static", StaticFiles(directory="static"),
name="static") templates = Jinja2Templates(directory="templates")
 app.include_router(auth.router, prefix="/api")
app.include_router(student.router, prefix="/api")
app.include_router(admin.router)

@app.on_event("startup") async
def startup_event():
    # Check if admin user exists, if not create one     admin_email =
"admin@college.edu"     existing_admin = await
db["users"].find_one({"email": admin_email})     if not existing_admin:
admin_user = {
            "full_name": "System Administrator",
            "register_number": "ADMIN001",
            "email": admin_email,
            "department": "ALL",
            "year": 0,
            "semester": 0,
            "hashed_password": get_password_hash("admin123"),
            "role": "admin",
            "created_at": datetime.utcnow()
        }         await db["users"].insert_one(admin_user)
print(f"Created default admin user: {admin_email} / admin123")

# --- Frontend Routes ---
 async def get_current_user_optional(request:
Request):
    token = request.cookies.get("access_token")
if token:         try:
            import jwt             from routers.auth import SECRET_KEY,
ALGORITHM             payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])             user = await
db["users"].find_one({"email": payload.get("sub")})             return user
except:             return None     return None
```

```
@app.get("/", response_class=HTMLResponse) async
def read_root(request: Request):
    user = await get_current_user_optional(request)
if user:         if user["role"] == "admin":
            return RedirectResponse(url="/admin/dashboard")
else:
            return RedirectResponse(url="/dashboard")     return
templates.TemplateResponse("index.html", {"request": request})

@app.get("/login", response_class=HTMLResponse) async
def login_page(request: Request):
    user = await get_current_user_optional(request)
if user:         if user["role"] == "admin":
            return RedirectResponse(url="/admin/dashboard")
return RedirectResponse(url="/dashboard")     return
templates.TemplateResponse("login.html", {"request": request})

@app.get("/register", response_class=HTMLResponse) async
def register_page(request: Request):
    user = await get_current_user_optional(request)
if user:
        return RedirectResponse(url="/dashboard")     return
templates.TemplateResponse("register.html", {"request": request})

@app.get("/dashboard", response_class=HTMLResponse) async
def dashboard_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("dashboard.html", {"request": request,
"user": user})

@app.get("/profile", response_class=HTMLResponse) async
def profile_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("profile.html", {"request": request,
"user": user})

@app.get("/skills", response_class=HTMLResponse) async
def skills_page(request: Request):
    user = await get_current_user_optional(request)
if not user or user["role"] != "student":
return RedirectResponse(url="/login")     return
templates.TemplateResponse("skills.html", {"request":
request,
"user": user})

@app.get("/assessment", response_class=HTMLResponse) async
def assessment_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("assessment.html", {"request": request,
"user": user})
```

```
@app.get("/courses", response_class=HTMLResponse) async
def courses_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("courses.html", {"request": request,
"user": user})

@app.get("/recommendations", response_class=HTMLResponse) async def
recommendations_page(request: Request):     user = await
get_current_user_optional(request)     if not user or user["role"] !=
"student":         return RedirectResponse(url="/login")     return
templates.TemplateResponse("recommendations.html", {"request":
request, "user": user})

@app.get("/progress", response_class=HTMLResponse) async
def progress_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("progress.html", {"request": request,
"user": user})

@app.get("/achievements", response_class=HTMLResponse) async
def achievements_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("achievements.html", {"request": request,
"user": user})

@app.get("/report", response_class=HTMLResponse) async
def report_page(request: Request):
    user = await get_current_user_optional(request)     if not user or
user["role"] != "student":         return RedirectResponse(url="/login")
return templates.TemplateResponse("report.html", {"request": request,
"user": user})

@app.get("/admin/dashboard", response_class=HTMLResponse)
async def admin_dashboard_page(request: Request):
user = await get_current_user_optional(request)
    if not user or user["role"] != "admin":         return
RedirectResponse(url="/login")     return
templates.TemplateResponse("admin.html", {"request": request,
"user": user})
 if __name__ ==
"__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

File: database/database.py

```
import os from motor.motor_asyncio import
AsyncIOMotorClient from dotenv import load_dotenv
 load_dotenv()

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/") DATABASE_NAME
= os.getenv("DATABASE_NAME", "student_skills_db")
 client =
AsyncIOMotorClient(MONGO_URI) db =
client[DATABASE_NAME]
 def
get_db():
return db
```

File: routers/auth.py

```
from fastapi import APIRouter, Depends, HTTPException, status, Response,
Request from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm from passlib.context import CryptContext from
datetime import datetime, timedelta import jwt from jwt.exceptions import
InvalidTokenError from database.database import get_db from models.user
import UserCreate, Token, TokenData import os  router =
APIRouter(prefix="/auth", tags=["auth"])
 pwd_context = CryptContext(schemes=["bcrypt"],
deprecated="auto") oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="auth/login")

SECRET_KEY = os.getenv("SECRET_KEY",
"supersecretkey_student_skills_management_2026")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES",
"120"))  def verify_password(plain_password,
hashed_password):
    return pwd_context.verify(plain_password, hashed_password)
 def
get_password_hash(password):
    return pwd_context.hash(password)
 def create_access_token(data: dict, expires_delta: timedelta | None =
None):
    to_encode = data.copy()
if expires_delta:
        expire = datetime.utcnow() + expires_delta
else:
        expire = datetime.utcnow() + timedelta(minutes=15)
to_encode.update({"exp": expire})     encoded_jwt =
jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)     return
encoded_jwt
 async def get_current_user(request: Request,
db=Depends(get_db)):
    token = request.cookies.get("access_token")
if not token:
        # Check authorization header if not in cookies
auth_header = request.headers.get("Authorization")         if
auth_header and auth_header.startswith("Bearer "):
            token = auth_header.split(" ")[1]
         if not
token:
        raise HTTPException(
status_code=status.HTTP_401_UNAUTHORIZED,             detail="Not
authenticated",             headers={"WWW-Authenticate":
"Bearer"},
        )

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
email: str = payload.get("sub")         role: str = payload.get("role")
if email is None:
            raise credentials_exception
token_data = TokenData(email=email, role=role)
except InvalidTokenError:         raise HTTPException(
```

```
status_code=status.HTTP_401_UNAUTHORIZED,
detail="Could not validate credentials",
headers={"WWW-Authenticate": "Bearer"},
        )          user = await db["users"].find_one({"email":
token_data.email})     if user is None:
        raise HTTPException(status_code=404, detail="User not found")
return user

async def get_current_active_student(current_user: dict =
Depends(get_current_user)):
    if current_user.get("role") != "student":
        raise HTTPException(status_code=400, detail="Not enough permissions")
return current_user

async def get_current_active_admin(current_user: dict =
Depends(get_current_user)):
    if current_user.get("role") != "admin":
        raise HTTPException(status_code=400, detail="Not enough permissions")
return current_user

@router.post("/register") async def register_user(user: UserCreate,
db=Depends(get_db)):     # Check if user already exists
existing_user = await db["users"].find_one({"email": user.email})
if existing_user:
        raise HTTPException(status_code=400, detail="Email already registered")
         existing_reg = await
db["users"].find_one({"register_number":
user.register_number})
if existing_reg:
        raise HTTPException(status_code=400, detail="Register number already
registered")          user_dict = user.model_dump()
user_dict["hashed_password"] = get_password_hash(user_dict.pop("password"))
user_dict["created_at"] = datetime.utcnow()
         await
db["users"].insert_one(user_dict)

    # Create empty profile for the student
if user.role == "student":         profile
= {
            "email": user.email,
            "full_name": user.full_name,
            "register_number": user.register_number,
            "department": user.department,
            "year": user.year,
            "semester": user.semester,
            "career_goal": "Software Developer", # Default for screenshots
            "interests": ["Coding", "Technology"],
            "profile_image": "default.png"
        }         await
db["student_profiles"].insert_one(profile)

        # Auto-seed some initial data for presentation/screenshots
import random         # 1. Seed Skills         initial_skills = [
            {"email": user.email, "skill_name": "Python", "category":
"Technical", "current_level": "Intermediate", "target_level": "Expert",
```

```
"progress_percentage": 75, "status": "In Progress"},
            {"email": user.email, "skill_name": "Communication", "category":
"Soft", "current_level": "Beginner", "target_level": "Advanced",
"progress_percentage": 40, "status": "In Progress"},
            {"email": user.email, "skill_name": "Java", "category":
"Technical", "current_level": "Beginner", "target_level": "Intermediate",
"progress_percentage": 25, "status": "In Progress"}
        ]         await
db["student_skills"].insert_many(initial_skills)

        # 2. Seed an Assessment         await
db["assessment_results"].insert_one({
            "email": user.email,
            "skill_name": "Python",
            "difficulty": "Beginner",
            "score": 8,
            "total_questions": 10,
            "percentage": 80,
            "achieved_level": "Intermediate",
            "date": datetime.utcnow() - timedelta(days=5)
        })

        # 3. Seed a Course         await
db["course_progress"].insert_one({
            "email": user.email,
            "course_name": "Python Programming Masterclass",
            "status": "In Progress",
            "updated_at": datetime.utcnow()
        })

        # 4. Seed an Achievement         await
db["student_achievements"].insert_one({
            "email": user.email,
            "title": "First Skill Added",
            "description": "Add your first skill to the platform",
            "date_earned": datetime.utcnow() - timedelta(days=2)
        })      return {"message": "User registered
successfully"} @router.post("/login") async def
login_for_access_token(response: Response, form_data:
OAuth2PasswordRequestForm = Depends(), db=Depends(get_db)):     user
= await db["users"].find_one({"email": form_data.username})     if
not user or not verify_password(form_data.password,
user["hashed_password"]):         raise HTTPException(
status_code=status.HTTP_401_UNAUTHORIZED,
detail="Incorrect email or password",             headers={"WWW-
Authenticate": "Bearer"},
        )          access_token_expires =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)     access_token =
create_access_token(         data={"sub": user["email"], "role":
user["role"]}, expires_delta=access_token_expires
    )          response.set_cookie(
key="access_token",          value=access_token,
httponly=True,
max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
expires=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
```

```
samesite="lax",          secure=False # Set true
for HTTPS
    )          return {"access_token": access_token, "token_type":
"bearer", "role":
user["role"]}

@router.post("/logout") async def
logout(response: Response):
    response.delete_cookie("access_token")
return {"message": "Logged out successfully"}
```

File: routers/student.py

```
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, Form
from database.database import get_db from routers.auth import
get_current_active_student import os import shutil from typing import
Optional from datetime import datetime
 router =
APIRouter(tags=["student"])

@router.get("/dashboard/stats")
async def get_dashboard_stats(current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
email = current_user["email"]

    # Get total skills     student_skills = await
db["student_skills"].find({"email":
email}).to_list(length=100)     total_skills = len(student_skills)
completed_skills = sum(1 for s in student_skills if s.get("status") ==
"Completed")     in_progress_skills = sum(1 for s in student_skills if
s.get("status") ==
"In Progress")

    # Get assessments     assessments_completed = await
db["assessment_results"].count_documents({"email": email})

    # Get courses     courses_completed = await
db["course_progress"].count_documents({"email":
email, "status": "Completed"})

    # Get achievements     achievements_unlocked = await
db["student_achievements"].count_documents({"email": email})

    # Chart data (Aggregating progress by skill category or taking top 5
skills)     skill_labels = [s["skill_name"] for s in student_skills[:5]]
skill_data = [s["progress_percentage"] for s in student_skills[:5]]

return {
        "total_skills": total_skills,
        "completed_skills": completed_skills,
        "in_progress_skills": in_progress_skills,
        "assessments": assessments_completed,
        "courses": courses_completed,
```

```
        "achievements": achievements_unlocked,
        "skill_labels": skill_labels,
        "skill_data": skill_data
    }

@router.get("/profile")
async def get_profile(current_user: dict = Depends(get_current_active_student),
db=Depends(get_db)):     profile = await
db["student_profiles"].find_one({"email":
current_user["email"]}, {"_id": 0})
if not profile:
        # Create a default profile if missing
profile = {
            "email": current_user["email"],
            "full_name": current_user.get("full_name", "Student"),
            "register_number": current_user.get("register_number", ""),
            "department": current_user.get("department", "CSE"),
            "year": current_user.get("year", 1),
            "semester": current_user.get("semester", 1),
            "career_goal": "",
            "interests": [],
            "profile_image": "default.png"
        }         await
db["student_profiles"].insert_one(profile.copy())
        # Remove _id for JSON serialization
if "_id" in profile:             del
profile["_id"]     return profile

@router.post("/profile") async def update_profile(
full_name: str = Form(...),     department: str = Form(...),
year: int = Form(...),     semester: int = Form(...),
career_goal: str = Form(""),     interests: str = Form(""),
profile_image: Optional[UploadFile] = File(None),
current_user: dict = Depends(get_current_active_student),
db=Depends(get_db)
):     email =
current_user["email"]

update_data = {
        "full_name": full_name,
        "department": department,
        "year": year,
        "semester": semester,
        "career_goal": career_goal,
        "interests": [i.strip() for i in interests.split(",") if i.strip()]
    }          if profile_image:         # Save image
upload_dir = "static/images/profiles"
os.makedirs(upload_dir, exist_ok=True)         file_extension
= profile_image.filename.split(".")[-1]         filename =
f"{email.replace('@', '_').replace('.',
'_')}.{file_extension}"         filepath =
os.path.join(upload_dir, filename)         with
open(filepath, "wb") as buffer:
            shutil.copyfileobj(profile_image.file, buffer)
```

```
update_data["profile_image"] = filename
            await
db["student_profiles"].update_one(
        {"email": email},
        {"$set": update_data}
    )

    # Also update full_name in users collection
await db["users"].update_one(
        {"email": email},
        {"$set": {"full_name": full_name}}
    )          return {"message": "Profile updated
successfully"}

# --- Skills Management ---
 from pydantic import
BaseModel from bson import
ObjectId
 class
SkillCreate(BaseModel):
    skill_name: str
category: str
current_level: str
target_level: str
progress_percentage: int
status: str

@router.get("/skills") async def get_skills(current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
    email = current_user["email"]     skills = await
db["student_skills"].find({"email":
email}).to_list(length=100)     for
skill in skills:         skill["id"] =
str(skill["_id"])         del
skill["_id"]     return skills

@router.post("/skills")
async def add_skill(skill: SkillCreate, current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
email = current_user["email"]

    # Prevent duplicate     existing = await
db["student_skills"].find_one({"email": email,
"skill_name": skill.skill_name})
if existing:
        raise HTTPException(status_code=400, detail="Skill already added")
             skill_dict =
skill.model_dump()
skill_dict["email"] = email
result = await
db["student_skills"].insert_one(ski
ll_dict)     return {"message":
"Skill added successfully", "id":
str(result.inserted_id)}
```

```
@router.put("/skills/{skill_id}")
async def update_skill(skill_id: str, skill: SkillCreate, current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
try:
        obj_id = ObjectId(skill_id)     except:         raise
HTTPException(status_code=400, detail="Invalid ID format")
             result = await
db["student_skills"].update_one(
        {"_id": obj_id, "email": current_user["email"]},
        {"$set": skill.model_dump()}
    )          if
result.modified_count == 0:
        raise HTTPException(status_code=404, detail="Skill not found or no
changes made")              return {"message": "Skill updated
successfully"}

@router.delete("/skills/{skill_id}")
async def delete_skill(skill_id: str, current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
try:
        obj_id = ObjectId(skill_id)     except:         raise
HTTPException(status_code=400, detail="Invalid ID format")
             result = await db["student_skills"].delete_one({"_id": obj_id,
"email":
current_user["email"]})     if
result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Skill not found")
             return {"message": "Skill deleted
successfully"}

# --- Assessments ---
 class
AssessmentSubmit(BaseModel):
    skill_name: str
difficulty: str     score:
int     total_questions:
int

@router.get("/assessment/questions/{skill_name}/{difficulty}") async
def get_assessment_questions(skill_name: str, difficulty: str,
db=Depends(get_db)):
    # Realistic mock questions for screenshots
questions = []     if skill_name == "Python":
        questions = [
            {"id": 1, "question": f"Which of the following is used to define a
function in Python?", "options": ["def", "function", "func", "define"],
"answer": "def"},
            {"id": 2, "question": f"What is the output of print(2 ** 3)?",
"options": ["6", "8", "9", "12"], "answer": "8"},
            {"id": 3, "question": f"Which data structure is immutable in
Python?", "options": ["List", "Dictionary", "Set", "Tuple"], "answer":
"Tuple"},
            {"id": 4, "question": f"How do you insert an item at a given
position in a list?", "options": ["insert()", "append()", "add()", "push()"],
```

```
"answer": "insert()"},
            {"id": 5, "question": f"What keyword is used for exception
handling?", "options": ["catch", "except", "error", "throw"], "answer":
"except"}
        ] * 2 # Duplicate to make 10 questions
elif skill_name == "Java":
        questions = [
            {"id": 1, "question": f"Which of these is not a Java keyword?",
"options": ["class", "interface", "extends", "implement"], "answer":
"implement"},
            {"id": 2, "question": f"What is the size of an int variable in
Java?", "options": ["8 bit", "16 bit", "32 bit", "64 bit"], "answer": "32 bit"}
        ] * 5
else:
        # Generic realistic questions
for i in range(1, 11):
questions.append({
                "id": i,
                "question": f"What is a core concept of {skill_name} at the
{difficulty} level?",
                "options": ["Syntax formatting", "Memory management",
"Objectoriented design", "Algorithm optimization"],
                "answer": "Object-oriented design"
            })

    # Ensure they have sequential IDs and there are exactly 10
final_q = []     for i, q in enumerate(questions[:10]):
        new_q = q.copy()
new_q["id"] = i + 1
final_q.append(new_q)
             return {"questions":
final_q}

@router.post("/assessment/submit")
async def submit_assessment(data: AssessmentSubmit, current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
    percentage = int((data.score / data.total_questions) * 100)
         level = "Beginner"     if percentage >= 80 and
data.difficulty == "Advanced":
        level = "Expert"     elif percentage >= 60 and data.difficulty in
["Advanced", "Intermediate"]:
        level = "Advanced"
elif percentage >= 50:
level = "Intermediate"
result_doc = {
        "email": current_user["email"],
        "skill_name": data.skill_name,
        "difficulty": data.difficulty,
        "score": data.score,
        "total_questions": data.total_questions,
        "percentage": percentage,
        "achieved_level": level,
        "date": datetime.utcnow()
    }          await
db["assessment_results"].insert_one(result_doc)
```

```
    # Check achievements
if percentage == 100:
        await check_and_award_achievement(current_user["email"], "Assessment
Expert", "Score 100% in any assessment", db)
             return {"message": "Assessment submitted", "percentage":
percentage,
"level": level}

# --- Recommendations & Gap Analysis ---
 def
get_required_skills(career_goal):
    # Mock data for required skills based on career goal
requirements = {
        "AI Engineer": {"Python": 90, "Machine Learning": 85, "SQL": 70,
"Communication": 70},
        "Software Developer": {"Java": 85, "Data Structures": 80, "Problem
Solving": 80, "Git/GitHub": 75},
        "Data Scientist": {"Python": 90, "SQL": 85, "Machine Learning": 70,
"Presentation Skills": 75},
        "DevOps Engineer": {"Cloud Computing": 85, "Python": 75, "Git/GitHub":
90, "Communication": 70},
        "Cyber Security Analyst": {"Python": 80, "SQL": 75, "Problem Solving":
85, "Communication": 80},
        "Full Stack Developer": {"JavaScript": 90, "HTML/CSS": 85, "SQL": 80,
"Git/GitHub": 80}
    }
    return requirements.get(career_goal, {"Communication": 60, "Problem
Solving": 60})

@router.get("/recommendations")
async def get_recommendations(current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
email = current_user["email"]
         profile = await db["student_profiles"].find_one({"email":
email})     career_goal = profile.get("career_goal", "")
         if not
career_goal:
        return {"gaps": [], "recommendations": [], "message": "Please set a
career goal in your profile first."}

    required_skills = get_required_skills(career_goal)
student_skills_cursor = db["student_skills"].find({"email": email})
student_skills = {s["skill_name"]: s["progress_percentage"] for s in await
student_skills_cursor.to_list(length=100)}
         gaps = []
recommendations = []
         for req_skill, req_percent in
required_skills.items():         current_percent =
student_skills.get(req_skill, 0)         gap = req_percent
- current_percent
                 status_text = "Requirement
achieved"          if gap > 0:
            status_text = f"Need {gap}% improvement"
recommendations.append({
```

```
                "title": f"Improve {req_skill}",
                "current": current_percent,
                "target": req_percent,
                "description": f"Focus on {req_skill} to reach your goal of
becoming a {career_goal}."
            })
gaps.append({
            "skill": req_skill,
            "current": current_percent,
            "required": req_percent,
            "gap": gap if gap > 0 else 0,
            "status": status_text
        })              return {"gaps": gaps, "recommendations":
recommendations, "career_goal":
career_goal}

# --- Courses ---
 class
CourseProgressUpdate(BaseModel):
    course_name: str
status: str

@router.get("/courses") async def get_courses(current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
    email = current_user["email"]

    # Dummy courses
all_courses = [
        {"id": "c1", "name": "Python Programming Masterclass", "skill":
"Python", "difficulty": "Beginner", "duration": "10h"},
        {"id": "c2", "name": "Machine Learning Fundamentals", "skill": "Machine
Learning", "difficulty": "Intermediate", "duration": "15h"},
        {"id": "c3", "name": "Advanced SQL Queries", "skill": "SQL",
"difficulty": "Advanced", "duration": "8h"},
        {"id": "c4", "name": "Effective Communication", "skill":
"Communication", "difficulty": "Beginner", "duration": "5h"},
        {"id": "c5", "name": "Data Structures in Java", "skill": "Java",
"difficulty": "Intermediate", "duration": "20h"}
    ]          progress = await
db["course_progress"].find({"email":
email}).to_list(length=100)     prog_dict = {p["course_name"]:
p["status"] for p in progress}
         for c in
all_courses:
        c["status"] = prog_dict.get(c["name"], "Not Started")

    return all_courses

@router.post("/course/progress") async def update_course_progress(data:
CourseProgressUpdate, current_user: dict
= Depends(get_current_active_student), db=Depends(get_db)):
email = current_user["email"]
         await
db["course_progress"].update_one(
        {"email": email, "course_name": data.course_name},
```

```
        {"$set": {"status": data.status, "updated_at": datetime.utcnow()}},
upsert=True
    )          if data.status == "Completed":         await
check_and_award_achievement(email, "Learning Champion",
"Completed a learning course", db)
             return {"message": "Progress
updated"}
 async  def  check_and_award_achievement(email,  title,  description,  db):
existing = await db["student_achievements"].find_one({"email": email,
"title": title})
if not existing:
        await db["student_achievements"].insert_one({
            "email": email,
            "title": title,
            "description": description,
            "date_earned": datetime.utcnow()
        })

# --- Progress & Achievements ---

@router.get("/progress/stats")
async def get_progress_stats(current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):
    email = current_user["email"]     skills = await
db["student_skills"].find({"email":
email}).to_list(length=100)

    # Calculate mock historical progress based on current average     if not
skills:         return {"historical": {"months": [], "data": []}, "radar":
{"labels":
[], "data": []}}

    current_avg = sum(s["progress_percentage"] for s in skills) / len(skills)

    # Generate historical data leading up to current avg based on actual
current month     current_month = datetime.utcnow().month     month_names
= ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug",
"Sep", "Oct", "Nov", "Dec"]
months = []     for i in
range(4, -1, -1):
        months.append(month_names[(current_month - 1 - i) % 12])
             hist_data = [max(0, current_avg - 30), max(0, current_avg -
20), max(0, current_avg - 10), max(0, current_avg - 5), current_avg]

    # Radar chart data for top 5 skills     radar_labels =
[s["skill_name"] for s in skills[:6]]     radar_data =
[s["progress_percentage"] for s in skills[:6]]

return {
        "historical": {"months": months, "data": hist_data},
"radar": {"labels": radar_labels, "data": radar_data}
    }

@router.get("/achievements/list") async def
get_achievements(current_user: dict =
```

```
Depends(get_current_active_student), db=Depends(get_db)):
email = current_user["email"]

    # All possible badges
all_badges = [
        {"title": "First Skill Added", "description": "Add your first skill to
the platform"},
        {"title": "Assessment Expert", "description": "Score 100% in any
assessment"},
        {"title": "Learning Champion", "description": "Completed a learning
course"},
        {"title": "5 Skills Tracked", "description": "Add 5 skills to your
profile"},
        {"title": "Python Beginner", "description": "Complete Python Beginner
Assessment"}
    ]          earned = await
db["student_achievements"].find({"email":
email}).to_list(length=100)     earned_titles = {e["title"]:
e["date_earned"] for e in earned}
         result = []
for b in all_badges:
        b["earned"] = b["title"] in earned_titles         if
b["earned"]:             b["date"] =
earned_titles[b["title"]].strftime("%Y-%m-%d")
result.append(b)

return result
@router.get("/rep
ort/data")
async def get_report_data(current_user: dict =
Depends(get_current_active_student), db=Depends(get_db)):     email =
current_user["email"]     profile = await
db["student_profiles"].find_one({"email": email}, {"_id":
0})     skills = await db["student_skills"].find({"email": email},
{"_id":
0}).to_list(length=100)     assessments = await
db["assessment_results"].find({"email": email}, {"_id":
0}).to_list(length=100)     achievements = await
db["student_achievements"].find({"email": email},
{"_id": 0}).to_list(length=100)     courses = await
db["course_progress"].find({"email": email}, {"_id":
0}).to_list(length=100)
         overall_score = 0     if skills:         overall_score =
sum(s["progress_percentage"] for s in skills) / len(skills)
return {
        "profile": profile,
        "skills": skills,
        "assessments": assessments,
        "achievements": achievements,
        "courses": courses,
        "overall_score": int(overall_score)
    }
```

File: routers/admin.py

```
from fastapi import APIRouter, Depends, HTTPException
from database.database import get_db from
routers.auth import get_current_active_admin from
collections import defaultdict
 router = APIRouter(prefix="/api/admin",
tags=["admin"])

@router.get("/stats")
async def get_admin_stats(current_user: dict =
Depends(get_current_active_admin), db=Depends(get_db)):
    total_students = await db["users"].count_documents({"role": "student"})
total_skills_tracked = await db["student_skills"].count_documents({})
assessments_completed = await db["assessment_results"].count_documents({})
courses_completed = await db["course_progress"].count_documents({"status":
"Completed"})

    # Calculate average skill score     skills = await
db["student_skills"].find({}, {"progress_percentage":
1}).to_list(length=1000)     avg_score = 0     if skills:
avg_score = int(sum(s["progress_percentage"] for s in skills) /
len(skills))              return {
        "total_students": total_students,
        "total_skills_tracked": total_skills_tracked,
        "assessments_completed": assessments_completed,
        "courses_completed": courses_completed,
        "average_skill_score": avg_score,
        "active_students": total_students # Mocking active students
    }

@router.get("/analytics")
async def get_admin_analytics(current_user: dict =
Depends(get_current_active_admin), db=Depends(get_db)):
    # Department-wise skills     profiles = await
db["student_profiles"].find({}).to_list(length=1000)     dept_counts =
defaultdict(int)     for p in profiles:
        dept_counts[p.get("department", "Unknown")] += 1
             dept_labels =
list(dept_counts.keys())     dept_data =
list(dept_counts.values())

    # Popular skills     skills = await
db["student_skills"].find({}, {"skill_name":
1}).to_list(length=1000)
skill_counts = defaultdict(int)
for s in skills:
        skill_counts[s["skill_name"]] += 1

    # Sort and get top 5
    sorted_skills = sorted(skill_counts.items(), key=lambda x: x[1],
reverse=True)[:5]     popular_labels = [s[0] for s in sorted_skills]
popular_data = [s[1] for s in sorted_skills]
```

```
return {
        "departments": {"labels": dept_labels, "data": dept_data},
        "popular_skills": {"labels": popular_labels, "data": popular_data}
    }

@router.get("/students")
async def get_all_students(department: str = None, current_user: dict =
Depends(get_current_active_admin), db=Depends(get_db)):
query = {}     if department and department != "ALL":
query["department"] = department
            profiles = await
db["student_profiles"].find(query, {"_id":
0}).to_list(length=100)

    # Add stats to each student     for p in profiles:         email =
p["email"]         skill_count = await
db["student_skills"].count_documents({"email":
email})         assessment_count = await
db["assessment_results"].count_documents({"email": email})
p["total_skills"] = skill_count
p["assessments_completed"] = assessment_count
            return
profiles
```

File: models/user.py

```
from pydantic import BaseModel, EmailStr, Field
from typing import Optional from datetime
import datetime
 class
UserCreate(BaseModel):
full_name: str
register_number: str
email: EmailStr
department: str     year:
int     semester: int
password: str     role: str
= "student"
 class UserInDB(UserCreate):     hashed_password: str
created_at: datetime = Field(default_factory=datetime.utcnow)
 class
Token(BaseModel):
access_token: str
token_type: str
 class TokenData(BaseModel):
email: Optional[str] = None
role: Optional[str] = None
```

File: templates/index.html

```
{% extends "base.html" %}
```

```
{% block content %}
<!-- Hero Section -->
<section class="hero-section text-center">
    <div class="container">
        <h1 class="display-4 fw-bold mb-4">Build Skills. Track Progress. Shape
Your Future.</h1>
        <p class="lead mb-5">A smart platform for students to identify, develop
and monitor the skills required for academic and career success.</p>
        <div>
            <a href="/register" class="btn btn-light btn-lg text-primary me-3
fw-bold px-4">Get Started</a>
            <a href="/login" class="btn btn-outline-light btn-lg fw-bold px-
4">Student Login</a>
        </div>
    </div>
</section>

<!-- Features Section -->
<section id="features" class="py-5 bg-light">
    <div class="container py-5">
        <div class="text-center mb-5">
            <h2 class="fw-bold">Platform Features</h2>
            <p class="text-muted">Everything you need to accelerate your
career</p>
        </div>
        <div class="row g-4">
            <div class="col-md-4">
                <div class="glass-card p-4 h-100 text-center">
                    <div class="display-4 text-primary mb-3"><i class="fa-solid
fa-chart-line"></i></div>
                    <h4>Skill Gap Analysis</h4>
                    <p class="text-muted">Compare your current skills against
industry requirements and identify areas for improvement.</p>
                </div>
            </div>
            <div class="col-md-4">
                <div class="glass-card p-4 h-100 text-center">
                    <div class="display-4 text-success mb-3"><i class="fa-solid
fa-graduation-cap"></i></div>                    <h4>Assessments &
Learning</h4>
                    <p class="text-muted">Take quizzes to test your knowledge
and get personalized course recommendations.</p>
                </div>
            </div>
            <div class="col-md-4">
                <div class="glass-card p-4 h-100 text-center">
                    <div class="display-4 text-warning mb-3"><i class="fa-solid
fa-medal"></i></div>
                    <h4>Achievements</h4>
                    <p class="text-muted">Earn badges and track your progress
as you master new technical and soft skills.</p>
                </div>
            </div>
```

```
        </div>
    </div>
</section> {%
endblock %}
```

File: templates/student_base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Student Dashboard - SkillTrack{% endblock
%}</title>     <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/fontawesome/6.4.0/css/all.min.css"
>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&d
isplay=swap" rel="stylesheet">
    <link rel="stylesheet" href="/static/css/style.css?v=1.1">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    {% block extra_head %}{% endblock %}
    <style>
        .wrapper { display: flex; width: 100%; align-items: stretch; }

        /* Sidebar Styles */
#sidebar {             min-width:
250px;             max-width:
250px;             transition:
all 0.3s;             z-index:
1000;
        }

        #sidebar.active { margin-left: -250px; }

        #sidebar .sidebar-header {
padding: 20px;             font-weight:
700;             font-size: 1.2rem;
        }

        #sidebar ul p { color: #fff; padding: 10px; }
        #sidebar ul li a {             padding:
15px 20px;             font-size: 1rem;
display: block;             text-decoration:
none;             transition: 0.2s;
border-left: 3px solid transparent;
        }
        #sidebar ul li a i { margin-right: 10px; width: 20px; text-align:
center; }

        /* Content Styles */
#content {
width: 100%;
padding: 0;             min-
height: 100vh;
transition: all 0.3s;
        }
```

```
        .top-navbar {             padding:
15px 25px;             display: flex;
justify-content: space-between;
align-items: center;
        }

        .content-body { padding: 25px; }

        @media (max-width: 768px) {
            #sidebar { margin-left: -250px; }
            #sidebar.active { margin-left: 0; }
        }
    </style>
</head>
<body>
    <div class="wrapper">
        <!-- Sidebar -->
        <nav id="sidebar">
            <div class="sidebar-header">
                <i class="fa-solid fa-graduation-cap me-2"></i>SkillTrack
            </div>
            <ul class="list-unstyled components">
                <li class="{% if request.url.path == '/dashboard' %}active{%
endif %}">
                    <a href="/dashboard"><i class="fa-solid fa-chart-pie"></i>
Dashboard</a>
                </li>
                <li class="{% if request.url.path == '/profile' %}active{%
endif %}">
                    <a href="/profile"><i class="fa-solid fa-user"></i> My
Profile</a>
                </li>
                <li class="{% if request.url.path == '/skills' %}active{% endif
%}">
                    <a href="/skills"><i class="fa-solid fa-list-check"></i> My
Skills</a>
                </li>
                <li class="{% if request.url.path == '/assessment' %}active{%
endif %}">
                    <a href="/assessment"><i class="fa-solid fa-
clipboardquestion"></i> Skill Assessment</a>
                </li>
                <li class="{% if request.url.path == '/courses' %}active{%
endif %}">
                    <a href="/courses"><i class="fa-solid fa-book-open"></i>
Learning Courses</a>
                </li>
                <li class="{% if request.url.path == '/recommendations'
%}active{% endif %}">
                    <a href="/recommendations"><i class="fa-solid
falightbulb"></i> Recommendations</a>
                </li>
```

```
                <li class="{% if request.url.path == '/progress' %}active{%
endif %}">
                    <a href="/progress"><i class="fa-solid fa-chart-line"></i>
Progress</a>
                </li>
                <li class="{% if request.url.path == '/achievements' %}active{%
endif %}">
                    <a href="/achievements"><i class="fa-solid fa-medal"></i>
Achievements</a>
                </li>
                <li class="{% if request.url.path == '/report' %}active{% endif
%}">
                    <a href="/report"><i class="fa-solid fa-file-contract"></i>
Reports</a>
                </li>
            </ul>
        </nav>

        <!-- Page Content -->
        <div id="content">
            <!-- Top Navbar -->
            <div class="top-navbar">
                <button type="button" id="sidebarCollapse" class="btn
btnlight">
                    <i class="fa-solid fa-bars"></i>
                </button>
                <div class="d-flex align-items-center">
                    <span class="me-3 fw-medium">{{ user.full_name }}</span>
<button id="logoutBtn" class="btn btn-outline-danger btnsm">Logout</button>
</div>
            </div>

            <!-- Main Body -->
            <div class="content-body">
                {% block student_content %}{% endblock %}
            </div>
        </div>
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.
js"></script>     <script>
document.getElementById('sidebarCollapse').addEventListener('click',
function () {
document.getElementById('sidebar').classList.toggle('active');
        });
document.getElementById('logoutBtn').addEventListener('click', async ()
=> {             await fetch('/api/auth/logout', { method:
'POST' });             window.location.href = '/login';
        });
    </script>
    {% block extra_js %}{% endblock %}
</body>
</html>
```

File: templates/dashboard.html

```
{% extends "student_base.html" %}

{% block title %}Dashboard - SkillTrack{% endblock %}

{% block student_content %}
<div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="fw-bold">Welcome, {{ user.full_name }}!</h2>
    <a href="/profile" class="btn btn-primary btn-sm"><i class="fa-solid
fapen"></i> Complete Profile</a>
</div>

<!-- Dashboard Cards -->
<div class="row g-4 mb-5">
    <div class="col-md-4 col-lg-2">
        <div class="card border-0 shadow-sm text-center h-100 py-3">
            <h1 class="display-5 fw-bold text-primary" id="totalSkills">0</h1>
            <p class="text-muted mb-0">Total Skills</p>
        </div>
    </div>
    <div class="col-md-4 col-lg-2">
        <div class="card border-0 shadow-sm text-center h-100 py-3">
            <h1 class="display-5 fw-bold text-success"
id="completedSkills">0</h1>
            <p class="text-muted mb-0">Completed</p>
        </div>
    </div>
    <div class="col-md-4 col-lg-2">
        <div class="card border-0 shadow-sm text-center h-100 py-3">
            <h1 class="display-5 fw-bold text-warning"
id="inProgressSkills">0</h1>
            <p class="text-muted mb-0">In Progress</p>
        </div>
    </div>
    <div class="col-md-4 col-lg-2">
        <div class="card border-0 shadow-sm text-center h-100 py-3">
            <h1 class="display-5 fw-bold text-info"
id="assessmentsCompleted">0</h1>
            <p class="text-muted mb-0">Assessments</p>
        </div>
    </div>
    <div class="col-md-4 col-lg-2">
        <div class="card border-0 shadow-sm text-center h-100 py-3">
            <h1 class="display-5 fw-bold text-secondary"
id="coursesCompleted">0</h1>             <p class="text-
muted mb-0">Courses</p>
        </div>
    </div>
    <div class="col-md-4 col-lg-2">
        <div class="card border-0 shadow-sm text-center h-100 py-3">
            <h1 class="display-5 fw-bold text-danger"
id="achievementsUnlocked">0</h1>
            <p class="text-muted mb-0">Achievements</p>
        </div>
```

```
    </div>
</div>

<div class="row">
    <div class="col-lg-8 mb-4">
        <div class="card border-0 shadow-sm h-100">
            <div class="card-body">
                <h5 class="card-title fw-bold mb-4">Skill Progress
Overview</h5>
                <div class="chart-container">
                    <canvas id="skillProgressChart"></canvas>
                </div>
            </div>
        </div>
    </div>
    <div class="col-lg-4 mb-4">
        <div class="card border-0 shadow-sm h-100">
            <div class="card-body">
                <h5 class="card-title fw-bold mb-4">Recent Activity</h5>
                <div class="timeline" id="recentActivity">
                    <p class="text-muted">Loading activities...</p>
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}

{% block extra_js %}
<script>
    // Fetch dashboard data     async function loadDashboardData() {
try {             const response = await fetch('/api/dashboard/stats');
if (response.ok) {                 const data = await response.json();
document.getElementById('totalSkills').textContent = data.total_skills;
document.getElementById('completedSkills').textContent =
data.completed_skills;
document.getElementById('inProgressSkills').textContent =
data.in_progress_skills;
document.getElementById('assessmentsCompleted').textContent =
data.assessments;
document.getElementById('coursesCompleted').textContent = data.courses;
document.getElementById('achievementsUnlocked').textContent =
data.achievements;

                // Initialize chart
initChart(data.skill_labels, data.skill_data);
            }
        } catch (error) {             console.error("Error
loading dashboard data", error);
        }
    }      function initChart(labels, dataPoints) {
const ctx =
document.getElementById('skillProgressChart').getContext('2d');
```

```
        // Use dummy data if none provided (for demo purposes if DB is empty)
if (!labels || labels.length === 0) {             labels = ['Technical',
'Communication', 'Problem Solving',
'Leadership', 'Teamwork'];
dataPoints = [0, 0, 0, 0, 0];
        }          new Chart(ctx, {             type: 'bar',
data: {                 labels: labels,
datasets: [{                     label: 'Skill Progress (%)',
data: dataPoints,                     backgroundColor:
'rgba(79, 70, 229, 0.7)',                     borderColor:
'rgba(79, 70, 229, 1)',                     borderWidth: 1,
borderRadius: 4
                }]             },
options: {                 responsive:
true,                 maintainAspectRatio:
false,                  scales: {
y: {                         beginAtZero:
true,                         max: 100
                    }
                }
            }
        });
    }

    // Load data on page load
document.addEventListener('DOMContentLoaded', loadDashboardData);
</script>
{% endblock %}
```

File: templates/report.html

```
{% extends "student_base.html" %}

{% block title %}Skill Report - SkillTrack{% endblock %}

{% block student_content %}
<div class="d-flex justify-content-between align-items-center mb-4">
    <h3 class="fw-bold">Comprehensive Skill Report</h3>
    <button class="btn btn-primary" onclick="window.print()"><i class="fa-solid
fa-print me-2"></i>Download / Print Report</button>
</div>

<div class="card border-0 shadow-sm" id="reportCard">
    <div class="card-body p-5">
        <div class="text-center mb-5">
            <i class="fa-solid fa-graduation-cap display-4 text-primary mb-
3"></i>
            <h2 class="fw-bold">Student Skills Development Report</h2>
            <p class="text-muted">Generated by SkillTrack System</p>
        </div>

        <div id="loading" class="text-center py-5">
            <div class="spinner-border text-primary" role="status"></div>
```

```
        </div>

        <div id="reportContent" class="d-none">
            <!-- Profile Info -->
            <div class="row mb-4 bg-light p-4 rounded">
                <div class="col-md-6">
                    <p class="mb-1"><strong>Name:</strong> <span
id="rName"></span></p>
                    <p class="mb-1"><strong>Register Number:</strong> <span
id="rReg"></span></p>
                    <p class="mb-1"><strong>Email:</strong> <span
id="rEmail"></span></p>
                </div>
                <div class="col-md-6 text-md-end">
                    <p class="mb-1"><strong>Department:</strong> <span
id="rDept"></span></p>
                    <p class="mb-1"><strong>Year/Sem:</strong> <span
id="rYear"></span></p>
                    <p class="mb-1"><strong>Career Goal:</strong> <span
id="rGoal"></span></p>                </div>
            </div>

            <div class="row text-center mb-5">
                <div class="col-4">
                    <h1 class="text-primary fw-bold" id="rScore">0</h1>
                    <p class="text-muted">Overall Skill Score</p>
                </div>
                <div class="col-4 border-start border-end">
                    <h1 class="text-success fw-bold" id="rSkillsCount">0</h1>
                    <p class="text-muted">Total Skills Tracked</p>
                </div>
                <div class="col-4">
                    <h1 class="text-warning fw-bold" id="rBadges">0</h1>
                    <p class="text-muted">Achievements Earned</p>
                </div>
            </div>

            <!-- Skills Table -->
            <h5 class="fw-bold mb-3 border-bottom pb-2">Skills Profile</h5>
            <div class="table-responsive mb-5">
                <table class="table table-bordered">
                    <thead class="table-light">
<tr>
                            <th>Skill Name</th>
                            <th>Category</th>
                            <th>Current Level</th>
                            <th>Target Level</th>
                            <th>Progress</th>
                            <th>Status</th>
                        </tr>
                    </thead>
                    <tbody id="rSkillsTable"></tbody>
                </table>
            </div>
```

```
            <!-- Assessments Table -->
            <h5 class="fw-bold mb-3 border-bottom pb-2">Assessment
Performance</h5>
            <div class="table-responsive mb-5">
                <table class="table table-bordered">
                    <thead class="table-light">
                        <tr>
                            <th>Skill</th>
                            <th>Difficulty</th>
<th>Score</th>
                            <th>Percentage</th>
                            <th>Achieved Level</th>
                            <th>Date</th>
                        </tr>
                    </thead>
                    <tbody id="rAssessmentsTable"></tbody>
                </table>
            </div>

            <div class="text-center text-muted mt-5 pt-3 border-top">
                <small>This report is auto-generated by the Student Skills
Development Management System.</small>
            </div>
        </div>
    </div>
</div>
{% endblock %}

{% block extra_js %}
<script>
    async function loadReport() {         try {
const res = await fetch('/api/report/data');
const data = await res.json();

document.getElementById('loading').classList.add('d-none');
document.getElementById('reportContent').classList.remove('dnone');
                        // Profile
document.getElementById('rName').textContent = data.profile.full_name;
document.getElementById('rReg').textContent = data.profile.register_number;
document.getElementById('rEmail').textContent = data.profile.email;
document.getElementById('rDept').textContent = data.profile.department;
document.getElementById('rYear').textContent = `Year
${data.profile.year}, Sem ${data.profile.semester}`;
document.getElementById('rGoal').textContent = data.profile.career_goal || 'Not
specified';
                        // Stats
document.getElementById('rScore').textContent = data.overall_score;
document.getElementById('rSkillsCount').textContent = data.skills.length;
document.getElementById('rBadges').textContent = data.achievements.length;

            // Skills Table             const skillsBody =
document.getElementById('rSkillsTable');             if (data.skills.length
=== 0) {                 skillsBody.innerHTML = '<tr><td colspan="6"
class="text-center text-muted">No skills tracked yet.</td></tr>';
```

```
            } else {
data.skills.forEach(s => {
skillsBody.innerHTML += `
                        <tr>
                            <td>${s.skill_name}</td>
                            <td>${s.category}</td>
                            <td>${s.current_level}</td>
<td>${s.target_level}</td>
                            <td>
                                <div class="progress" style="height: 10px;">
<div class="progress-bar bg-primary" style="width:
${s.progress_percentage}%"></div>
                                </div>
                                <small class="text-muted d-block
textcenter">${s.progress_percentage}%</small>
                            </td>
                            <td>${s.status}</td>
                        </tr>
                    `;
                });
            }

            // Assessments Table             const assBody =
document.getElementById('rAssessmentsTable');             if
(data.assessments.length === 0) {                 assBody.innerHTML =
'<tr><td colspan="6" class="text-center text-muted">No assessments taken
yet.</td></tr>';
            } else {                 data.assessments.forEach(a => {
const dateObj = new Date(a.date);                     const
formattedDate = dateObj.toLocaleDateString();
assBody.innerHTML += `
                        <tr>
                            <td>${a.skill_name}</td>
                            <td>${a.difficulty}</td>
                            <td>${a.score}/${a.total_questions}</td>
                            <td>${a.percentage}%</td>
<td><span class="badge
bgsuccess">${a.achieved_level}</span></td>
                            <td>${formattedDate}</td>
                        </tr>
                    `;
                });
            }

        } catch (e) { console.error(e); }
    }     document.addEventListener('DOMContentLoaded',
loadReport);
</script>
<style>
    @media print {
        #sidebar, .top-navbar, button { display: none !important; }
        #content { margin: 0 !important; width: 100% !important; padding: 0
!important; }
        .wrapper { background: white; }
        #reportCard { box-shadow: none !important; }
```

```
    }
</style>
{% endblock %}
```